

Generative Adversarial Networks (GANs)

CSCI 4425/5425: Introduction to Deep Learning

Paras Khanal, Shokhina Jalilova, Vincent Sanchez

Introduction

- A **Generative Adversarial Network (GAN)** is a deep learning framework designed to **learn the underlying distribution of real data** and generate new, realistic samples.
- It consists of two neural networks:
 - **Generator (G)**: creates synthetic data from random noise, producing a learned distribution $p_g(x)$
 - **Discriminator (D)**: evaluates whether a given sample is real or generated
- Adversarial setting / Competitive
 - The Generator learns to produce increasingly realistic data
 - The Discriminator learns to better distinguish real from fake images



Generator



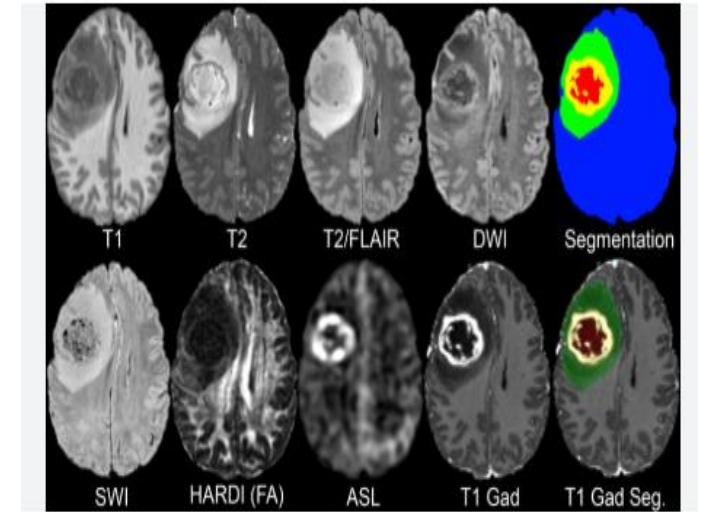
POLICE
Discriminator

Why ?

- GAN is **not a classification** or prediction model.
 - It is a **generative model**. The generator learns the real data distribution.
 - It generates the data for specific purposes.
- **Data Augmentation**
 - Generating additional training samples
 - Medical image, biology or rare datasets
- **Image generation**
 - Realistic images
 - Art, design, simulation
- **Image to Image translation; super resolution**
 - Color, format, sketch -> photo
 - Low resolution -> high resolution
- **Synthetic Data creation**
 - Healthcare
 - Autonomous Driving
 - Simulation environments

Real world problem – small dataset problem

- Limited labeled medical images for **rare disease**
- Some disease are rare. Examples include specific variants of pneumonia, COVID-19, brain tumor, etc.
- Medical data is **highly sensitive and restricted** (HIPAA regulation and others)
- Data collection is **very slow and expensive**
- Deep learning model typically require thousands or millions of sample images
- Rare disease have few hundred images usually

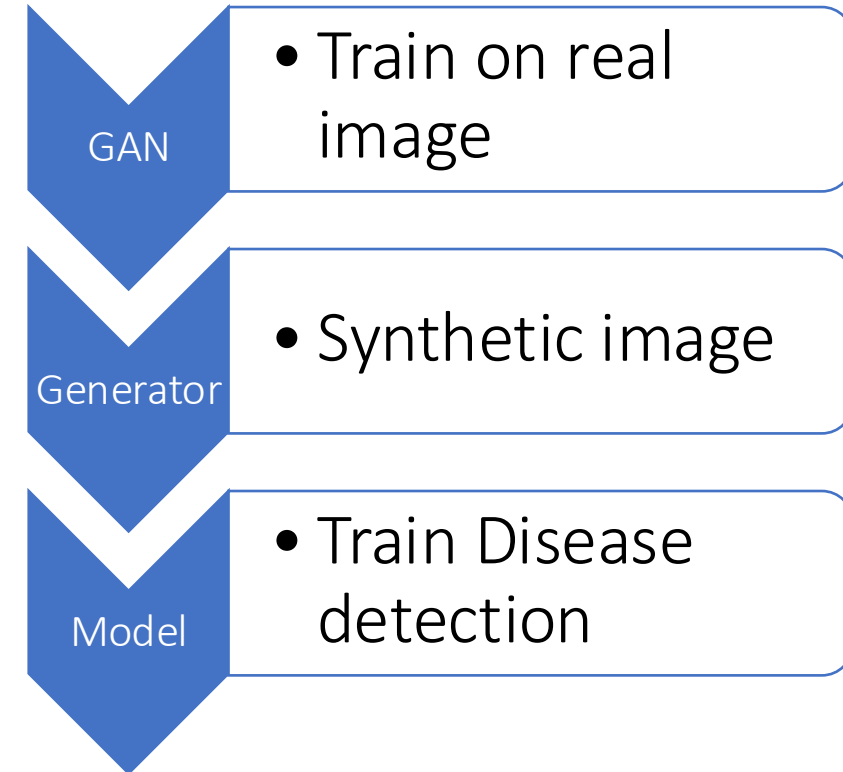


GAN is the solution

- GAN solves this by **learning data distribution and generating** additional samples like magic.
- GANs can generate synthetic medical images – **very similar** to real images

Example workflow:

1. **Train** a GAN on real medical images (e.g. MRI, CT Scans)
2. Generator creates **synthetic** images -> added to dataset
3. A disease **detection model** is trained with more data, improving accuracy.





How GAN compares to other models:

GAN vs VAE

Both models generate new samples but differ in training.

- **GAN**

- Uses adversarial training
- Produces sharper images
- Training can be unstable

- **VAE**

- Uses probabilistic latent variables
- Easier to train
- Generated samples often blurrier
- VAE optimizes a likelihood-based objective, while GAN minimizes distribution difference via adversarial learning.

GAN vs Diffusion Models

- **GAN**
 - Fast image generation
 - Hard to train
 - Risk of mode collapse
- **Diffusion**
 - Stable training
 - Higher image quality
 - Slower generation due to iterative steps
 - Diffusion models gradually remove noise from data.

GAN vs GNN

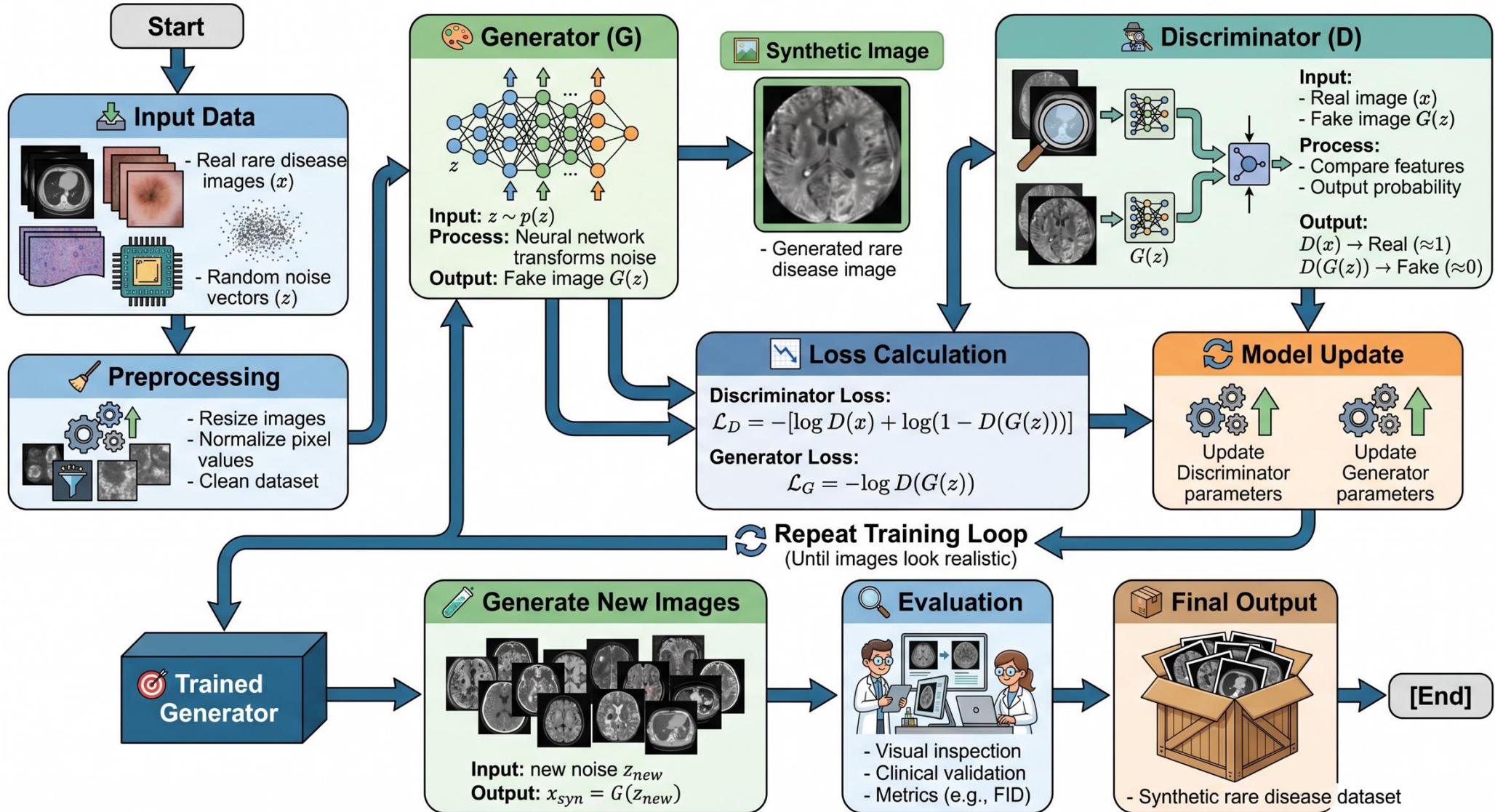
- **GAN**
 - Generates new data (continuous)
 - Focuses on continuous data generation
 - Use case: medical image generation, simulation
- **GNN**
 - Learn relationships in data (Graph)
 - Focuses on learning relationships through nodes
 - Use case: Social networks, recommendation



How does this magic happen: Basic Training process - outline

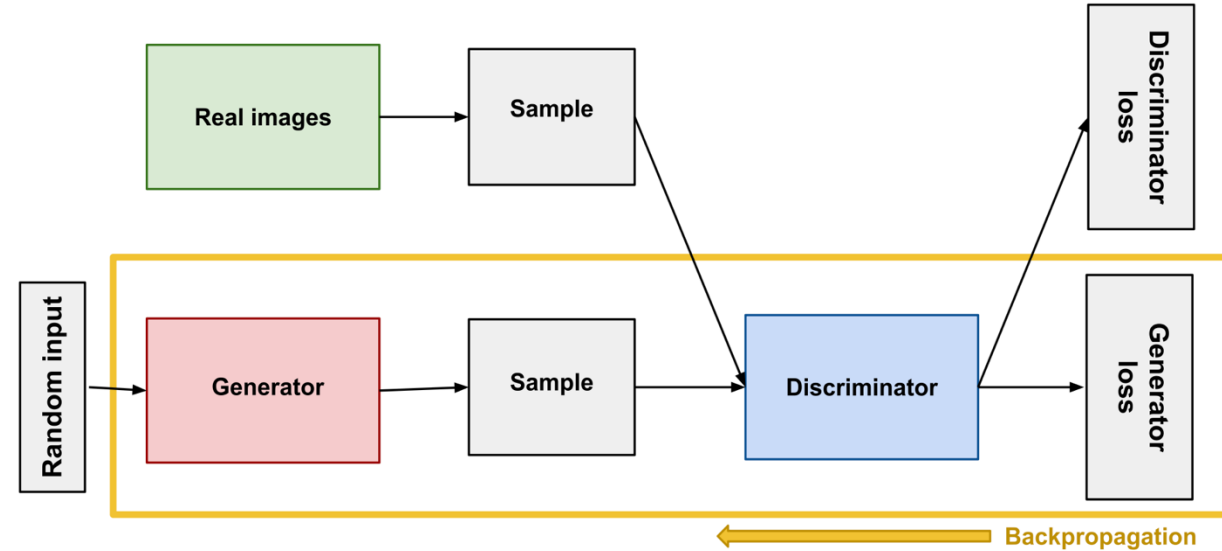
Component	Role (What it does)
Generator (G)	Creates synthetic images from random noise (z)
Discriminator (D)	Distinguishes between real images and generated images
Loss Calculation	Measures how well D detects and how well G fools D
Generator Update	Improves G to produce more realistic images
Discriminator Update	Improves D to better detect fake images
Iterative Training	Repeats the process through multiple training cycles
Optimization Goal	Minimax game: G minimizes, D maximizes the objective
Final Outcome	Generated data distribution $\approx$ real data distribution

GAN Architecture for Rare disease



The Generator

- Goal: find parameters θ so that generated samples are indistinguishable from real data
- A new sample x^* is generated in two steps:
 - Sample a latent noise vector z from a simple base distribution
 - $z \sim N(0, I)$ — standard normal, easy to sample from
 - Pass z through the generator network to get a fake sample:
 - $x^* = g[z, \theta]$
- The generator never sees real images directly — it only learns from the gradient signal fed back by the discriminator
- After training: discriminator is discarded; only $g[z, \theta]$ is kept for generating new samples

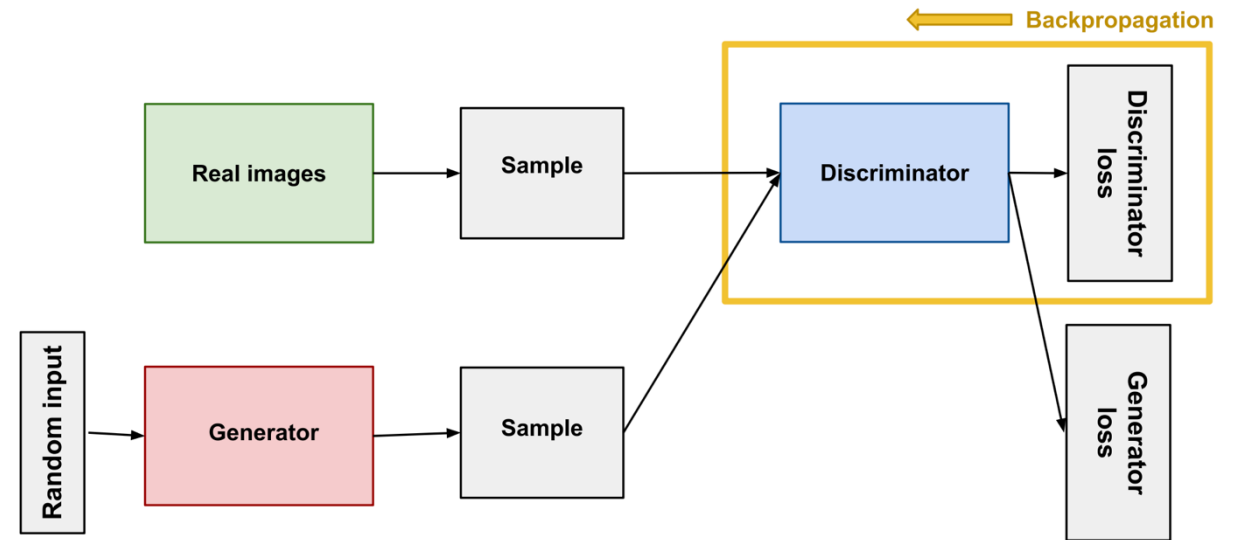


Why $z \sim N(0, I)$?

- Simple to sample from — any random number generator
- Low-dimensional (e.g., 100D) — the generator learns to warp this into the high-dimensional image space
- Continuous and smooth — nearby z values produce similar images (latent space interpolation)

The Discriminator

- The discriminator $f[x, \phi]$ takes any image x (real or fake) and outputs a single scalar
- Output passed through sigmoid to get a probability:
 - $\text{sig}[f[x, \phi]] \in [0, 1]$
 - High output (≈ 1) $\rightarrow$ D believes the input is real
 - Low output (≈ 0) $\rightarrow$ D believes the input is fake
- Why sigmoid? It maps any real number to a probability — essential for binary classification
- GAN labeling convention:
 - Real images: label $y = 1 \rightarrow$ D should output ≈ 1
 - Generated samples: label $y = 0 \rightarrow$ D should output ≈ 0
- At convergence: $\text{sig}[f[x, \phi]] = 0.5$ for all x — D is at chance (perfect uncertainty)



Why discard the discriminator after training?

- It is only a training signal — not the end product
- At convergence it outputs 0.5 everywhere, so it carries no useful information
- We only ever needed it to teach the generator

Notation

Symbol	Meaning
$\mathbf{x}$	A real data sample (e.g., an image)
$\mathbf{x}^*$	Generated (fake) sample
$\mathbf{z}$	Latent noise vector (input to generator)
$\Pr(\mathbf{x})$	True data distribution
$\Pr(\mathbf{x}^*)$	Generated sample distribution
$\mathbf{g}[\mathbf{z}, \theta]$	Generator network with parameters θ
$\mathbf{f}[\mathbf{x}, \phi]$	Discriminator network with parameters ϕ
$L[\varphi], L[\theta]$	Discriminator and generator loss functions
$\mathbb{E}[\cdot]$	Expectation (average over a distribution)
D_{JS}	Jensen–Shannon divergence
D_W	Wasserstein distance

Loss Function — overview

$$\min_G \max_D V(D, G)$$

$$V(D, G) = E_{x \sim p_{data}(x)}[\log D(x)] + E_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

Here:

- $D(x)$ = probability that real sample x is real
- $D(G(z))$ = probability that generated sample is real
- E = expectation over the data distribution

This objective theoretically minimizes the difference between the two distributions using Jensen–Shannon Divergence, encouraging the generator to produce realistic samples.

- The loss function in GAN measures how well the generator and discriminator perform their competing tasks.
- The discriminator's objective is to correctly classify real images as real and generated images as fake, while the generator's objective is to fool the discriminator into labeling fake images as real.
- This adversarial objective is typically expressed as a **minimax function**, where the discriminator maximizes the probability of correct classification and the generator minimizes the discriminator's ability to detect fake samples.

GAN loss function — setup

- Start from binary cross-entropy loss (section 5.4):

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_i -(1 - y_i) \log [1 - \operatorname{sig}[f[\mathbf{x}_i, \phi]]] - y_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$

- Assign real examples $y = 1$, generated samples $y = 0$:

$$\hat{\phi} = \operatorname{argmin}_{\phi} \left[\sum_j -\log [1 - \operatorname{sig}[f[\mathbf{x}_j^*, \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right]$$

- Substitute generator $\mathbf{x}_j^* = g[\mathbf{z}_j, \theta]$ (note: generator must maximize this loss to fool D):

$$\hat{\theta} = \operatorname{argmax}_{\theta} \left[\min_{\phi} \left[\sum_j -\log [1 - \operatorname{sig}[f[g[\mathbf{z}_j, \theta], \phi]]] - \sum_i \log [\operatorname{sig}[f[\mathbf{x}_i, \phi]]] \right] \right]$$

Equation 15.4 — the minimax GAN objective

GAN loss function — training the two networks

- Equation 15.4 is split into two loss functions for practical training:

Discriminator loss (minimize to update ϕ):

$$L[\phi] = \sum_j -\log[1 - \text{sig}[f[g[z_j, \theta], \phi]]] - \sum_i \log[\text{sig}[f[x_i, \phi]]]$$

Generator loss (minimize to update θ):

$$L[\theta] = \sum_j \log[1 - \text{sig}[f[g[z_j, \theta], \phi]]]$$

- Minimizing $L[\phi]$ trains the discriminator to correctly classify real vs. fake
- Minimizing $L[\theta]$ trains the generator to fool the discriminator
- At each step: draw batch of z , pass through G to get x^* , draw batch of real x_i , then update both

Training Process:

- In a **Generative Adversarial Network (GAN)**, training involves two neural networks learning simultaneously in an adversarial setup.
- The **generator** takes a random noise vector and produces synthetic (fake) images that resemble the training data, while the **discriminator** receives both real images from the dataset and fake images from the generator and attempts to classify them as real or fake.
- During each training iteration, the discriminator first learns to better distinguish real from generated images. Then the generator is updated based on the discriminator's feedback so it can produce more realistic outputs.
- This alternating process continues repeatedly until the generated data becomes increasingly difficult for the discriminator to distinguish from real data.

Training process - pseudocode

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log (1 - D(G(z^{(i)}))).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Training loop:

Generator →

Fake Image →

Discriminator →

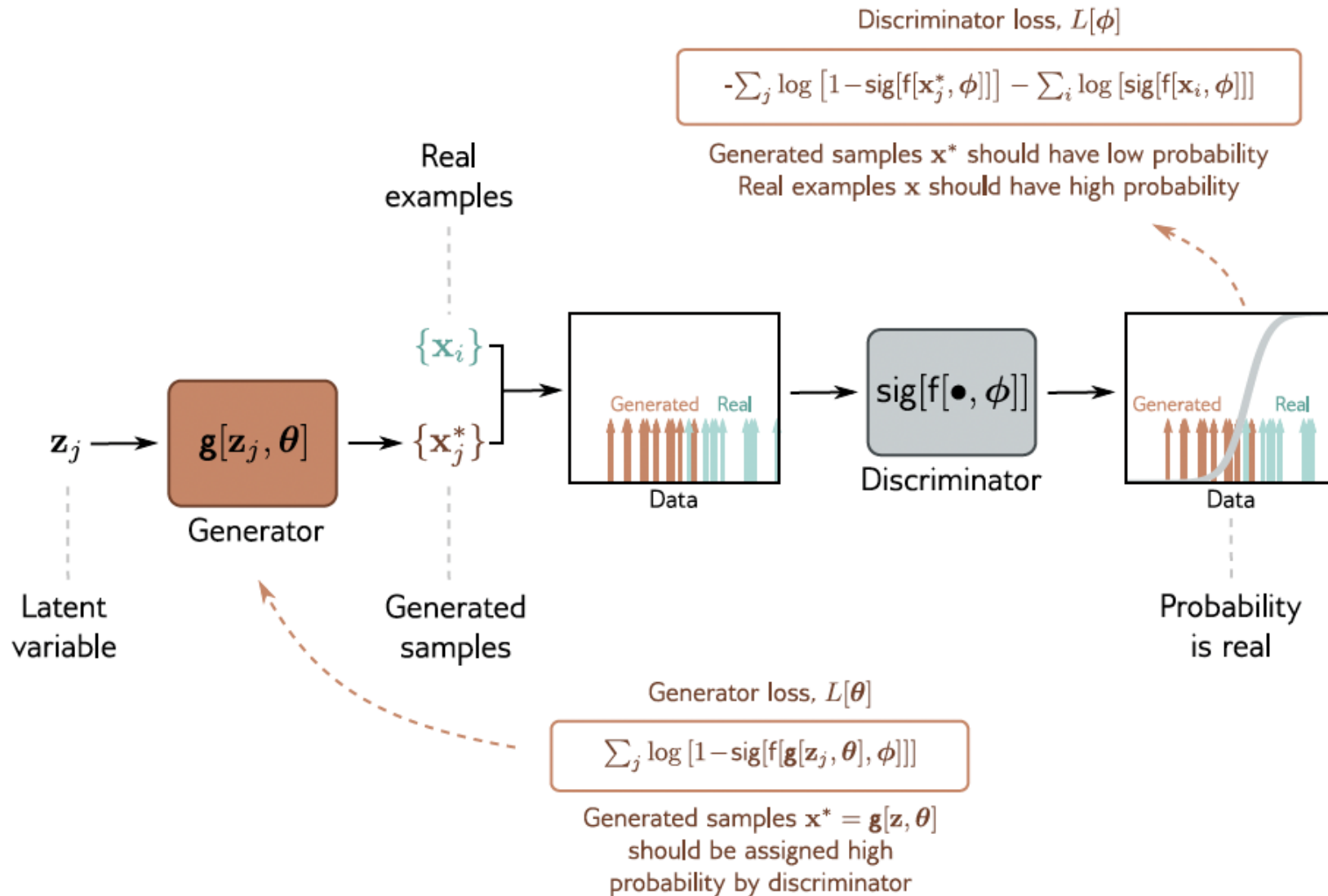
Real/Fake →

Loss →

Backpropagation →

Update weights

GAN training loop



- Alternate: update D for one or more steps, then update G for one step
- **Goal: Nash equilibrium** — neither player can unilaterally improve
- **Convergence: $D(x) = \frac{1}{2}$ for all x — perfect uncertainty**

Nash equilibrium in GAN training

- Technically, the solution to GAN training is a Nash equilibrium
- The optimization algorithm searches for a position that is simultaneously:
 - A minimum of $L[\varphi]$ — discriminator cannot improve further
 - A maximum of the generator objective — generator cannot improve further
- Upon convergence: $g[z, \theta]$ generates samples from the same distribution as the data
- Upon convergence: $\text{sig}[f[\bullet, \varphi]] = 0.5$ everywhere — discriminator is at chance
- In practice: true Nash equilibria are hard to find; **GAN training can be unstable**

Improving Stability

Section 15.2 — Analysis, Vanishing Gradients

Analysis of GAN loss — expectation form

- Dividing the sums in eq. 15.3 by I, J (number of samples) gives an expectation form:

$$\begin{aligned} L[\phi] &= \frac{1}{J} \sum_{j=1}^J \left(\log \left[1 - \text{sig}[f[\mathbf{x}_j^*, \phi]] \right] \right) + \frac{1}{I} \sum_{i=1}^I \left(\log \left[\text{sig}[f[\mathbf{x}_i, \phi]] \right] \right) \quad (15.6) \\ &\approx \mathbb{E}_{\mathbf{x}^*} \left[\log \left[1 - \text{sig}[f[\mathbf{x}^*, \phi]] \right] \right] + \mathbb{E}_{\mathbf{x}} \left[\log \left[\text{sig}[f[\mathbf{x}, \phi]] \right] \right] \\ &= \int Pr(\mathbf{x}^*) \log \left[1 - \text{sig}[f[\mathbf{x}^*, \phi]] \right] d\mathbf{x}^* + \int Pr(\mathbf{x}) \log \left[\text{sig}[f[\mathbf{x}, \phi]] \right] d\mathbf{x}, \end{aligned}$$

- The optimal discriminator for any fixed generator is:

$$Pr(\mathbf{x} \text{ is real}) = \text{sig}[f[\mathbf{x}, \phi]] = \frac{Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})}. \quad (15.7)$$

Equation 15.7 — optimal discriminator as a function of the two distributions

- This means: the discriminator tells us the relative probability of $\mathbf{x}$ being real vs. generated

Jensen–Shannon divergence

- Substituting the optimal discriminator into $L[\phi]$ and simplifying:

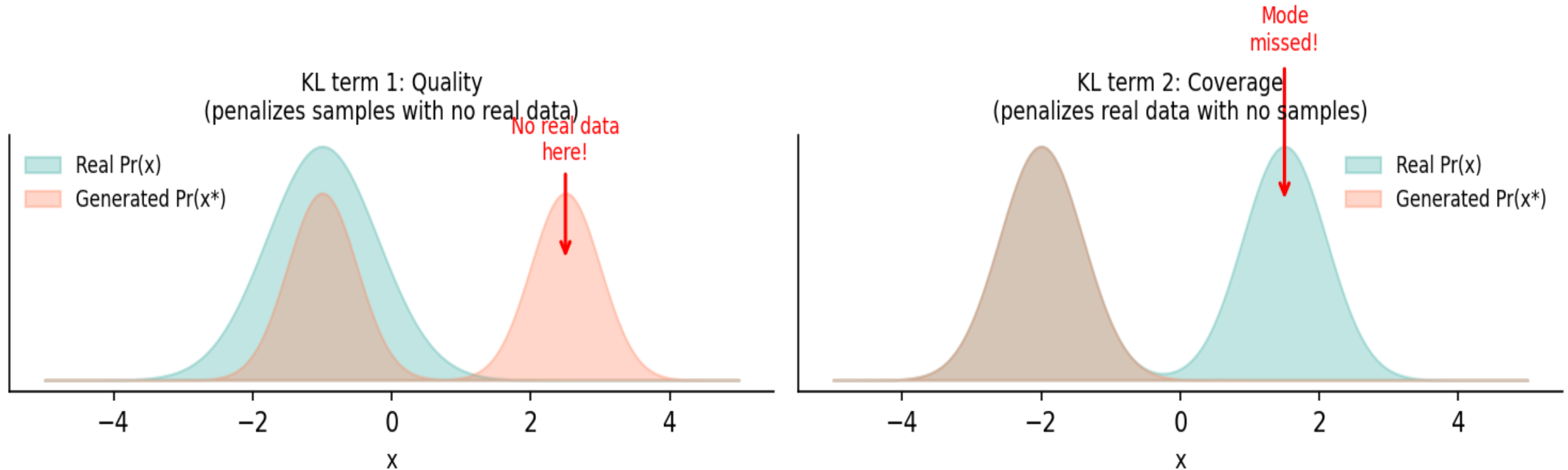
$$L[\phi] = \int Pr(\mathbf{x}^*) \log \left[\frac{Pr(\mathbf{x}^*)}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}^* + \int Pr(\mathbf{x}) \log \left[\frac{Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}.$$

- Ignoring additive/multiplicative constants, this is the Jensen–Shannon divergence:

$$D_{JS} [Pr(\mathbf{x}^*) || Pr(\mathbf{x})] = \underbrace{\frac{1}{2} \int Pr(\mathbf{x}^*) \log \left[\frac{2Pr(\mathbf{x}^*)}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}^*}_{\text{quality}} + \underbrace{\frac{1}{2} \int Pr(\mathbf{x}) \log \left[\frac{2Pr(\mathbf{x})}{Pr(\mathbf{x}^*) + Pr(\mathbf{x})} \right] d\mathbf{x}}_{\text{coverage}}.$$

- The two KL terms have interpretations:
 - First term (quality): penalizes generated samples in regions with no real data
 - Second term (coverage): penalizes real data regions with no generated samples
- The second term does not depend on the generator — GAN does not enforce coverage
→ mode dropping

Quality vs. coverage in the GAN loss

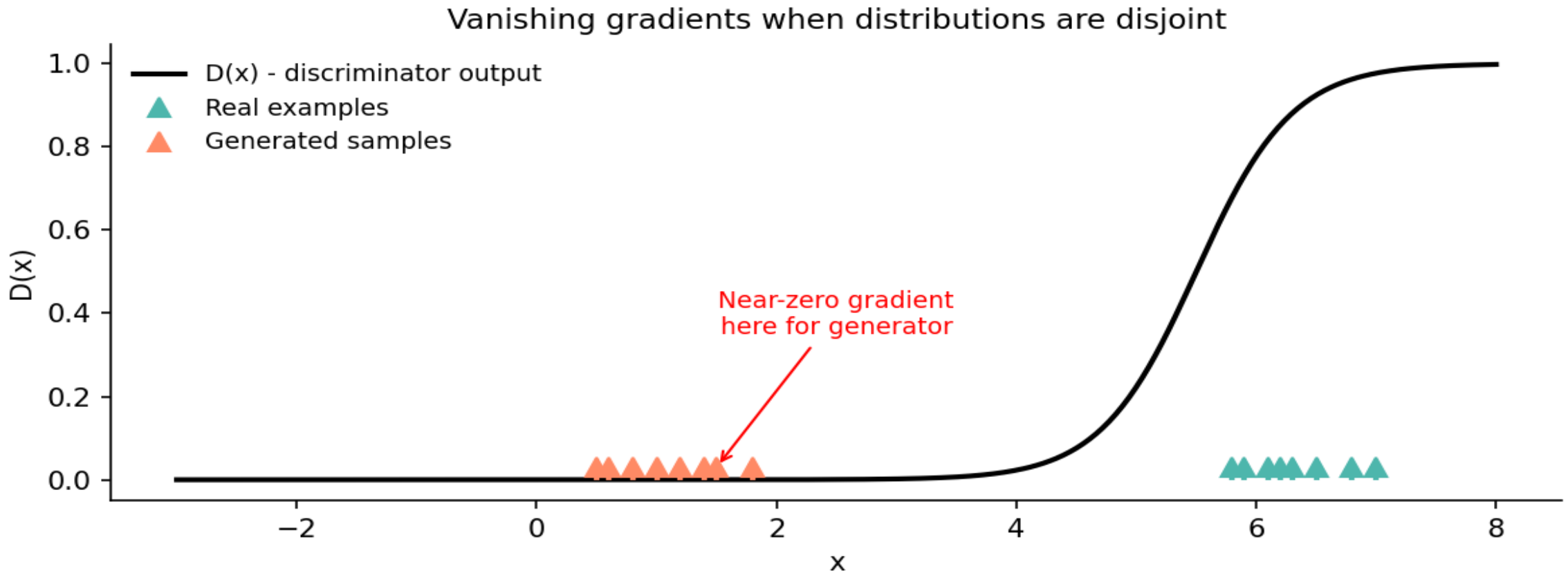


- The generator is trained on the quality term only — coverage is not penalized
- This explains why GANs are prone to mode dropping: they can "get away with" missing modes

Vanishing gradients

- Problem: if the two distributions $\Pr(x^*)$ and $\Pr(x)$ are disjoint, the JSD is $\log 2$ (a constant)
- Any small change to the generator will not decrease the loss — gradient is zero
- Why distributions may be disjoint:
 - Generated samples lie in a low-dimensional subspace (size of latent z)
 - Real examples also lie in a low-dimensional subspace (natural image manifold)
 - These subspaces may have little or no overlap
- Empirical evidence: if discriminator is trained further without updating generator, generator gradients vanish
- If the discriminator becomes too good — training of generator stalls

Vanishing gradients — illustration



- When samples are well-separated, the sigmoid discriminator is flat at generated sample positions
- Gradient to update generator parameters is near zero — training stalls

Code Example

Discriminator Update

```
def update_D(X, Z, net_D, net_G, loss, trainer_D):  
    """Update discriminator."""  
    batch_size = X.shape[0]  
    ones = torch.ones((batch_size,), device=X.device)  
    zeros = torch.zeros((batch_size,), device=X.device)  
    trainer_D.zero_grad()  
    real_Y = net_D(X)  
    fake_X = net_G(Z)  
    # Do not need to compute gradient for `net_G`, detach it from  
    # computing gradients.  
    fake_Y = net_D(fake_X.detach())  
    loss_D = (loss(real_Y, ones.reshape(real_Y.shape)) +  
              loss(fake_Y, zeros.reshape(fake_Y.shape))) / 2  
    loss_D.backward()  
    trainer_D.step()  
    return loss_D
```

- X -> real images
- Z -> random noise
- net_D -> discriminator
- net_G -> generator
- loss -> loss function
- trainer_D -> optimizer

Generator Update

```
def update_G(Z, net_D, net_G, loss, trainer_G):  
    """Update generator."""  
    batch_size = Z.shape[0]  
    ones = torch.ones((batch_size,), device=Z.device)  
    trainer_G.zero_grad()  
    # We could reuse `fake_X` from `update_D` to save computation  
    fake_X = net_G(Z)  
    # Recomputing `fake_Y` is needed since `net_D` is changed  
    fake_Y = net_D(fake_X)  
    loss_G = loss(fake_Y, ones.reshape(fake_Y.shape))  
    loss_G.backward()  
    trainer_G.step()  
    return loss_G
```

- Z -> random noise
 - net_G -> generator
 - net_D -> discriminator
 - loss -> loss function
 - trainer_G -> optimizer
-
- Key Difference:
 - Generator wants fake images labeled as 1

Training

```
def train(net_D, net_G, data_iter, num_epochs, lr_D, lr_G, latent_dim, data):
    loss = nn.BCEWithLogitsLoss(reduction='sum')
    for w in net_D.parameters():
        nn.init.normal_(w, 0, 0.02)
    for w in net_G.parameters():
        nn.init.normal_(w, 0, 0.02)
    trainer_D = torch.optim.Adam(net_D.parameters(), lr=lr_D)
    trainer_G = torch.optim.Adam(net_G.parameters(), lr=lr_G)
    animator = d2l.Animator(xlabel='epoch', ylabel='loss',
                           xlim=[1, num_epochs], nrows=2, figsize=(5, 5),
                           legend=['discriminator', 'generator'])
    animator.fig.subplots_adjust(hspace=0.3)
    for epoch in range(num_epochs):
        # Train one epoch
        timer = d2l.Timer()
        metric = d2l.Accumulator(3) # loss_D, loss_G, num_examples
        for (X,) in data_iter:
            batch_size = X.shape[0]
            Z = torch.normal(0, 1, size=(batch_size, latent_dim))
            metric.add(update_D(X, Z, net_D, net_G, loss, trainer_D),
                      update_G(Z, net_D, net_G, loss, trainer_G),
                      batch_size)
        # Visualize generated examples
        Z = torch.normal(0, 1, size=(100, latent_dim))
        fake_X = net_G(Z).detach().numpy()
        animator.axes[1].cla()
        animator.axes[1].scatter(data[:, 0], data[:, 1])
        animator.axes[1].scatter(fake_X[:, 0], fake_X[:, 1])
        animator.axes[1].legend(['real', 'generated'])
        # Show the losses
        loss_D, loss_G = metric[0]/metric[2], metric[1]/metric[2]
        animator.add(epoch + 1, (loss_D, loss_G))
    print(f'loss_D {loss_D:.3f}, loss_G {loss_G:.3f}, '
          f'{metric[2] / timer.stop():.1f} examples/sec')
```

- net_D -> discriminator
- net_G -> generator
- data_iter -> dataset iterator (training data)
- lr_D -> learning rate for discriminator
- lr_G -> learning rate for generator
- latent_dim -> size of the noise vector

Applications of GANs

- Image Generation
- Image-to-Image Translation
 - Sketch -> photo
 - Day -> night
- Style Transfer
 - Photo -> Van Gogh style painting
- Medical Imaging
 - Generate or enhance MRI/CT scans
 - Assist diagnosis with limited data

Downsides of GANs

- Unbalanced “Game”
 - If Discriminator gets too good: spots fakes too easily and Generator gets no useful feedback
 - If Generator gets too good: Discriminator can’t learn and the model collapses
- Mode Collapse
 - Generator finds one good output and repeats it over and over again
- Vanishing Gradients
 - When the Discriminator is confident, gradient becomes very small and Generator stops learning
- Poor architecture for images
 - Ignores pixels, edges, and shapes
 - Produces blurry or unrealistic images

GAN Family

Year	Model	Key Contribution
2014	GAN (Goodfellow et al.)	Introduced adversarial training (Generator vs Discriminator)
2015	DCGAN (Radford et al.)	Stable image generation with CNNs
2017	WGAN (Arjovsky et al.)	Improved stability using Wasserstein loss
2017	CycleGAN (Zhu et al.)	Image-to-image translation without paired data
2019	StyleGAN (Karras et al.)	Style-based generator for highly realistic images
2022+	GANs + Diffusion	Hybrid generative approaches combining GAN speed with diffusion quality

DCGAN (Deep Convolutional GAN)

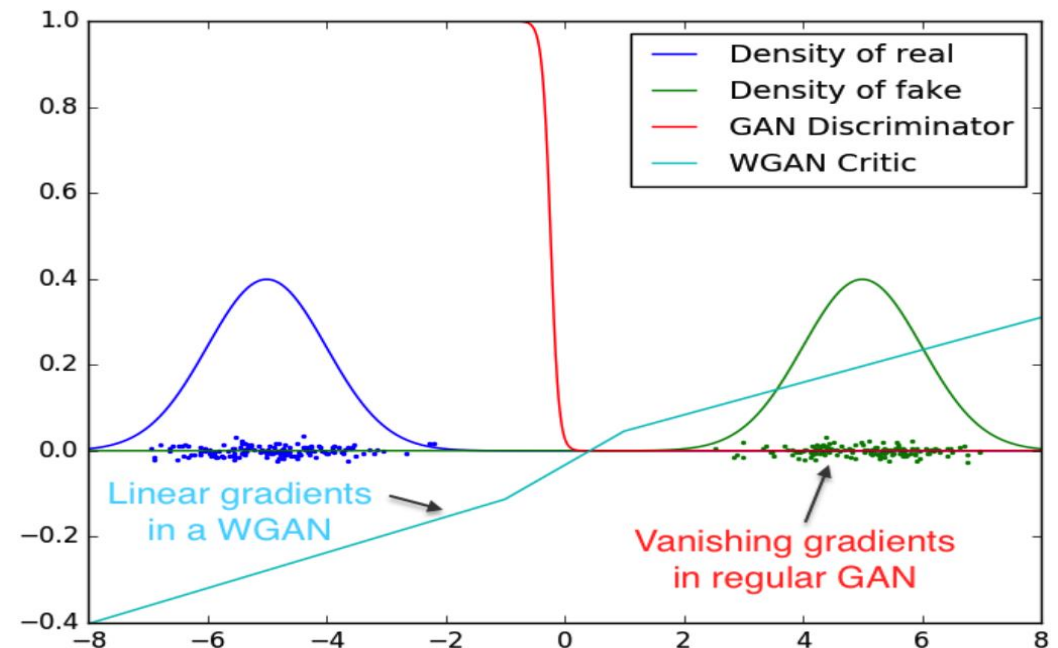
- Before DCGAN
 - Standard GAN uses fully connected layers
 - Very unstable with poor image quality
- What Changed?
 - Replaced fully connected layers with CNNs
- Why it matters
 - GANs become stable enough to train on more complex images
 - GANs were originally trained on simple datasets or even toy datasets
- Key design rules for stable training
 - No pooling -> uses stride convolutions instead
 - Uses Batch Normalization to stabilize training
- Activations
 - Generator -> ReLU (same as original)
 - Discriminator -> LeakyReLU (from sigmoid)

Introduction of LeakyReLU prevents dead neurons by allowing small gradients even for negative inputs

WGAN (Wasserstein GAN)

- Key Changes
 - Uses Wasserstein loss instead of cross-entropy
 - Removes sigmoid output from discriminator and changes to a “critic”
- Why it matters
 - Fixes the issue of vanishing gradients that occurred in original GAN
 - Generator receives useful gradients even when discriminator is strong
 - Loss now correlates with actual image quality

- Simplified Wasserstein Loss
$$L = \mathbb{E}[\text{critic}(\text{real})] - \mathbb{E}[\text{critic}(\text{fake})]$$



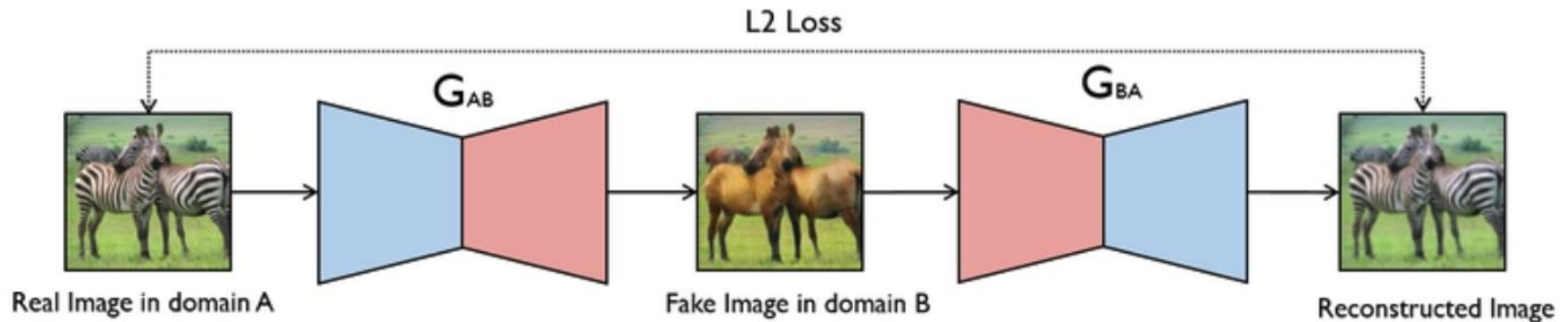
CycleGAN

- Key Ideas

- Learns translation WITHOUT paired data
- Uses two GANs for forward (A→B) and backward (B→A) mapping
- Doesn't create from noise but instead modifies an existing image

- Benefits

- No need for paired data
- Works on many domains
- Preserves structure using Cycle Consistency Loss



StyleGAN

- What Changed?
 - Traditional GANs generated images from noise
 - StyleGAN introduces a style-based generator architecture
- Layer-by-Layer Control:
 - Early layers -> coarse features (shape, pose)
 - Middle layers -> structure (facial features)
 - Late layers -> fine details (texture, color)
- Why it matters
 - Produces highly realistic images
 - Can modify specific features without affecting others

References

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Dive into deep learning. Cambridge University Press.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2020). Generative adversarial networks. *Communications of the ACM*, 63(11), 139-144.
- Radford, A., Metz, L., & Chintala, S. (2015). Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*.
- Arjovsky, M., Chintala, S., & Bottou, L. (2017, July). Wasserstein generative adversarial networks. In *International conference on machine learning* (pp. 214-223). Pmlr.
- Zhu, J. Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proceedings of the IEEE international conference on computer vision* (pp. 2223-2232).
- Karras, T., Laine, S., & Aila, T. (2019). A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 4401-4410).